Unidad II · Actividad 3

Gestión de roles y permisos

Fundamentos de Arquitectura de Software y Desarrollo Backend

El Sistema General de Ingresos ya tiene empresa, usuarios, cuentas, conceptos y movimientos. Esta unidad abre la gestión de accesos: quién puede hacer qué dentro de una empresa. El esquema GraphQL sigue siendo el mostrador. Lo que cambia es la regla que el resolver consulta antes de escribir.

Al terminar vas a poder distinguir un rol de un permiso, armar la relación de permisos por rol y proteger una operación que ya existe, como registrar un movimiento o crear una cuenta.

Antes de describir el sistema, conviene fijar el modelo de acceso: qué pregunta contesta un rol, qué otras formas existen y qué se gana y qué se pierde con cada una.

Cómo se controla el acceso

Cada vez que alguien pide una operación, el servidor contesta una pregunta antes de escribir: ¿esta acción está permitida para esta persona, sobre este recurso? Hay varias formas de guardar esa respuesta. Esta unidad usa la seguridad basada en roles. Las otras formas explican qué problema resuelve un rol y qué problema deja afuera.

Seguridad basada en roles

En la seguridad basada en roles (RBAC, *role-based access control*) la persona no acumula permisos sueltos. Recibe un rol, y el rol trae un conjunto de permisos.

Un permiso es una acción sobre un recurso: registrar un egreso, abrir una cuenta, leer los movimientos. Un rol es el puesto que agrupa esos permisos: capturista, contador, consulta. Ana entra como capturista. El servidor revisa el rol de esa membresía y, dentro de ese rol, si está el permiso de la operación.

La revisión recorre tres pasos:

- Quién actúa.
- Qué rol tiene esa persona en este contexto.
- Si ese rol incluye el permiso de la operación pedida.

Si el permiso está, la operación sigue. Si falta, se rechaza y no se escribe nada.

Hay dos variantes del mismo modelo. En la variante plana, cada rol tiene su propia lista. Si el administrador puede hacer lo mismo que el contador, esos permisos aparecen también en la lista del administrador. En la variante con herencia, un rol incluye los permisos de otro y la copia desaparece. La herencia ahorra filas. También hace que un cambio en el rol base modifique, a veces sin notarse, a todos los roles que lo incluyen. Esta actividad usa la variante plana: la lista de cada rol se lee sola.

Ventajas

- El negocio habla de puestos. "El contador abre cuentas" se traduce a un rol y a un permiso, con la misma regla para todas las personas de ese puesto.
- Un cambio de política se hace en el rol. Si mañana el capturista también puede abrir cuentas, se agrega ese permiso al rol y vale para todas las personas que lo tengan.
- Auditar es leer una lista. Los permisos del rol dicen qué puede hacer ese puesto.
- El catálogo se mantiene corto. Cien capturistas comparten una sola configuración.

Límites

- Dos personas con el mismo puesto y una excepción distinta no caben en el mismo rol. La salida fácil es crear otro rol por cada excepción. El catálogo crece y los nombres de puesto dejan de significar un trabajo claro. A eso se le llama explosión de roles.
- El rol nombra el puesto. No dice "solo los movimientos que tú capturaste" ni "solo si el monto es menor a 5 000". Esas condiciones hablan del recurso o del momento.
- Un catálogo de roles compartido aplica la misma política en todos lados. Una excepción local, "en esta empresa el capturista también abre cuentas", pide otro mecanismo.

Otras alternativas

Modelo	Dónde está la decisión	Ventaja	Límite
Permisos directos al usuario	Una lista de permisos por persona, sin rol de por medio	Cada persona puede tener una excepción exacta	Cincuenta capturistas son cincuenta copias. Cambiar la política obliga a recorrer a cada persona. "Qué puede un capturista" deja de tener una sola respuesta
Listas de control de acceso (ACL) y control discrecional (DAC)	Cada recurso guarda quién puede leerlo o modificarlo. Quien es dueño del recurso edita esa lista	Sirve para compartir un objeto concreto con una persona concreta, como un archivo que su dueño comparte	"Qué puede Ana en toda la empresa" obliga a leer la lista de cada recurso. El dueño puede conceder más de lo que la política de la empresa quería
Control obligatorio (MAC)	Una autoridad central etiqueta los datos y a las personas. La persona no regala un acceso, aunque haya creado la fila	La política no depende de lo que cada dueño decida conceder. Sirve cuando los datos tienen una clasificación	Hay que mantener etiquetas en cada fila y en cada persona. Para empresas que invitan a sus propios usuarios, operar eso cuesta más de lo que el problema pide
Control basado en atributos (ABAC)	Una regla mira atributos de la persona, del recurso, de la acción y, a veces, del contexto (monto, hora, empresa)	Una regla puede decir "el contador registra el egreso si el monto es menor a 10 000 y la cuenta es de su empresa", sin inventar un rol nuevo	Las reglas se vuelven otro programa. Explicar por qué una operación fue rechazada cuesta más, y cada operación acumula condiciones

Los permisos directos son útiles cuando las personas son pocas y la excepción es la norma. ACL y DAC se pueden usar cuando el permiso pertenece al objeto. MAC es suficiente cuando la clasificación del dato no se puede relajar. ABAC se puede usar cuando la decisión depende de un dato del recurso o del momento, y el nombre del puesto no la expresa.

Qué se usa en esta actividad

Esta actividad usa seguridad basada en roles, en la variante plana: un catálogo de roles, un catálogo de permisos y la relación entre los dos. La frontera entre empresas sigue siendo una regla de los datos, junto al rol. Las condiciones por monto, o la regla de "solo lo que yo capturé", quedan fuera de esta entrega.

El sistema que ya está construido

La empresa es la raíz. Cada empresa guarda sus datos aparte. Un usuario de la Empresa A no ve cuentas ni movimientos de la Empresa B.

```
Sistema
  ├── Empresa A
  │     ├── usuarios, con un administrador principal
  │     ├── cuentas
  │     ├── conceptos de ingreso y de egreso
  │     └── movimientos
  └── Empresa B
```

La membresía no vive dentro de `User`. Vive en `CompanyUser`, el puente que ya construiste:

```
Company  1 ——— N  CompanyUser  N ——— 1  User
```

`CompanyUser` responde tres preguntas de pertenencia: a qué empresa está unido el usuario, si esa membresía sigue activa y si es el administrador principal (`isAdmin`). La regla de oro se mantiene: una empresa tiene un solo administrador principal.

Encima de esa membresía ya operan las piezas del parcial anterior:

Pieza	Para qué sirve	Operaciones que ya existen
`Account`	Dónde se guarda el dinero	`createAccount`, `accounts`
`Concept`	Por qué entra o sale el dinero	`createConcept`, `concepts`
`AccountConcept`	Qué conceptos acepta cada cuenta	`assignConceptToAccount`
`Transaction`	El movimiento	`createTransaction`, `transactions`

Un movimiento suma si es `INCOME` y resta si es `EXPENSE`. El monto siempre es mayor que cero. La cuenta, el concepto y el usuario que captura pertenecen a la misma empresa. El concepto tiene que estar permitido en esa cuenta.

El servidor sigue siendo el que ya conoces: Python, Strawberry, GraphQL en `/graphql`, MySQL y, si el equipo lo dejó armado, Docker Compose para levantar la base y la API.

Por qué `isAdmin` no alcanza

`isAdmin` contesta una sola pregunta: ¿esta membresía es el administrador principal? Sirve para impedir un segundo administrador. No dice si esa persona puede crear cuentas, si solo captura movimientos o si únicamente consulta.

En una empresa real no todos hacen lo mismo:

Persona	Trabajo en el sistema
Administrador empresarial	Invita usuarios y gobierna la empresa
Contador	Abre cuentas, define conceptos y registra movimientos
Capturista	Registra ingresos y egresos. No cambia el catálogo
Consulta	Lee. No escribe

Si el resolver solo pregunta `isAdmin`, el capturista o no puede trabajar, o puede hacer todo. Los roles y los permisos separan esas dos situaciones.

En este sistema, esos tres nombres se reparten así.

Nombre	Pregunta	Ejemplo
Usuario	¿Quién es?	Ana, con su correo
Rol	¿Qué puesto tiene en esta empresa?	Contador
Permiso	¿Qué acción está permitida?	`transactions.write`

El usuario no recibe permisos sueltos. Recibe un rol. El rol trae los permisos. Cambiar lo que puede hacer un contador se hace una vez, en el rol, y aplica a todas las membresías que tengan ese rol.

1. Gestión de roles

Un rol es un puesto con nombre, dentro del sistema. No es una persona y no es una contraseña. En este sistema los roles son un catálogo compartido. La misma lista existe para todas las empresas. Lo que cambia de una empresa a otra es qué membresía recibe qué rol.

Cuatro roles cubren a los actores que ya identificaste:

Código	Nombre	Quién lo usa
`ADMINISTRADOR`	Administrador	El administrador principal de la empresa
`CONTADOR`	Contador	Quien arma cuentas y conceptos
`CAPTURISTA`	Capturista	Quien registra movimientos
`CONSULTA`	Consulta	Quien solo lee

El código es estable. El nombre se puede mostrar en pantalla. No uses el nombre como llave: `Contador` y `contador` se confundirían.

En GraphQL el rol se ve así:

```
type Role {
  id: ID!
  code: String!
  name: String!
  description: String
  isActive: Boolean!
}

input CreateRoleInput {
  code: String!
  name: String!
  description: String
}

extend type Query {
  roles(activeOnly: Boolean): [Role!]!
  role(id: ID!): Role
}

extend type Mutation {
  createRole(input: CreateRoleInput!): Role!
  deactivateRole(id: ID!): Role!
}
```

No hay borrado físico. Igual que la empresa, un rol se desactiva. Si mañana un movimiento quedó capturado por alguien con ese rol, el historial sigue teniendo a quién señalar.

Al arrancar, si la tabla está vacía, el servidor inserta los cuatro roles. Un reinicio no los duplica: solo inserta cuando `count()` es 0. Es el mismo cuidado que con los libros iniciales de la actividad anterior.

El administrador principal sigue naciendo con `createCompanyAdmin` y `isAdmin = true`. Esa regla no se sustituye. Además, esa membresía recibe el rol `ADMINISTRADOR`. `isAdmin` cuida que no haya dos administradores principales. El rol cuida lo que ese administrador puede ejecutar.

2. Gestión de permisos

Un permiso es una acción sobre una parte del sistema. Tampoco es una persona. El código se lee `recurso.accion`.

El catálogo sale de las operaciones que el sistema ya tiene. No inventes permisos para pantallas que todavía no existen.

Código	Acción
`company.users.read`	Ver las membresías de la empresa
`company.users.write`	Crear usuarios y desactivar membresías
`accounts.read`	Consultar cuentas
`accounts.write`	Crear cuentas
`concepts.read`	Consultar conceptos
`concepts.write`	Crear conceptos y asignarlos a una cuenta
`transactions.read`	Consultar movimientos
`transactions.write`	Registrar un ingreso o un egreso
`roles.read`	Ver roles y la matriz de permisos
`roles.write`	Asignar o quitar permisos de un rol, y asignar un rol a una membresía

```
type Permission {
  id: ID!
  code: String!
  name: String!
  description: String
  isActive: Boolean!
}

input CreatePermissionInput {
  code: String!
  name: String!
  description: String
}

extend type Query {
  permissions(activeOnly: Boolean): [Permission!]!
}

extend type Mutation {
  createPermission(input: CreatePermissionInput!): Permission!
}
```

El permiso, solo, no autoriza a nadie. Decir que existe `transactions.write` no le da esa acción al capturista. Esa decisión vive en la relación del siguiente apartado.

Dos permisos no se pisan por el nombre. `accounts.read` no incluye `accounts.write`. Quien escribe también necesita el de lectura si la pantalla consulta antes de guardar. En la matriz de abajo, los roles que escriben también leen.

3. Gestión de permisos por roles

Aquí se junta el puesto con la acción. Un rol tiene muchos permisos. Un permiso puede estar en muchos roles. Es la misma forma de `AccountConcept`: una tabla puente, no una columna repetida.

```
Role  N ——— role_permissions ——— N  Permission
```

```
type RolePermission {
  id: ID!
  role: Role!
```

```
  permission: Permission!
  isActive: Boolean!
}

input AssignPermissionToRoleInput {
  roleId: ID!
  permissionId: ID!
}

extend type Query {
  rolePermissions(roleId: ID!): [RolePermission!]!
}

extend type Mutation {
  assignPermissionToRole(input: AssignPermissionToRoleInput!): RolePermission!
  removePermissionFromRole(roleId: ID!, permissionId: ID!): Boolean!
}
```

`assignPermissionToRole` falla si esa pareja ya está activa. `removePermissionFromRole` desactiva la pareja. No borra la fila, para que el historial de la configuración siga ahí.

La configuración inicial de este sistema es esta matriz. Las filas son permisos. Las columnas son roles.

Permiso	ADMINISTRADOR	CONTADOR	CAPTURISTA	CONSULTA
`company.users.read`	sí	sí	no	sí
`company.users.write`	sí	no	no	no
`accounts.read`	sí	sí	sí	sí
`accounts.write`	sí	sí	no	no
`concepts.read`	sí	sí	sí	sí
`concepts.write`	sí	sí	no	no
`transactions.read`	sí	sí	sí	sí
`transactions.write`	sí	sí	sí	no
`roles.read`	sí	sí	no	no
`roles.write`	sí	no	no	no

Leída por rol:

- El administrador configura el acceso y administra usuarios.
- El contador arma el catálogo financiero y también registra movimientos. No administra usuarios ni cambia la matriz.
- El capturista registra movimientos y puede leer cuentas y conceptos para elegirlos. No los crea.
- Consulta lee la operación. No escribe.

Si mañana el contador también debe invitar usuarios, se agrega `company.users.write` a ese rol. No se recorre usuario por usuario.

Falta unir la membresía con el rol. `CompanyUser` sigue siendo el puente empresa-usuario. El rol no se guarda como texto dentro de `User`, por la misma razón por la que `companyId` no se guardó ahí: el puesto es de esa membresía, en esa empresa.

```
CompanyUser  1 ——— 1  CompanyUserRole  N ——— 1  Role
```

En esta actividad cada membresía activa tiene un solo rol activo. La tabla permite crecer después, pero la regla de ahora es una pareja activa.

```
type CompanyUserRole {
  id: ID!
  companyUser: CompanyUser!
  role: Role!
  isActive: Boolean!
}

input AssignRoleInput {
  companyUserId: ID!
  roleId: ID!
}

extend type Mutation {
  assignRole(input: AssignRoleInput!): CompanyUserRole!
}
```

`assignRole` desactiva el rol anterior de esa membresía y deja el nuevo activo. Así el capturista puede pasar a contador sin quedar con los dos puestos a la vez.

Dónde vive cada cosa

El patrón es el de las actividades de GraphQL con MySQL. El tipo GraphQL no es la tabla. El resolver traduce.

Pieza	Pregunta	En este tema
Tipo GraphQL	¿Qué pide el cliente?	`Role`, `Permission`, `RolePermission`
Modelo SQLAlchemy	¿Cómo se ve la tabla en Python?	`RoleModel`, `PermissionModel`, `RolePermissionModel`
Tabla MySQL	¿Dónde están las filas?	`roles`, `permissions`, `role_permissions`, `company_user_roles`

```
class RoleModel(Base):
    __tablename__ = "roles"

    id = Column(Integer, primary_key=True, index=True)
    code = Column(String(50), unique=True, nullable=False)
    name = Column(String(100), nullable=False)
    description = Column(String(255))
    is_active = Column(Boolean, nullable=False, default=True)
```

`PermissionModel` sigue la misma forma, con `code` único. `RolePermissionModel` guarda `role_id`, `permission_id` e `is_active`. `CompanyUserRoleModel` guarda `company_user_id`, `role_id` e `is_active`.

Las tablas se crean al arrancar con `Base.metadata.create_all` si el proyecto ya trabaja así. Si el proyecto usa Alembic, el cambio va en una migración nueva. `create_all` no borra filas que ya existen. La migración tampoco debe borrar cuentas ni movimientos.

El resolver es quien exige el permiso

GraphQL valida que el campo exista. No sabe si Ana es capturista. Esa pregunta ocurre en la ejecución, dentro del resolver, con una sesión de SQLAlchemy.

La cadena es corta:

```
membresía CompanyUser
  → rol activo
    → permisos activos de ese rol
```

```
      → ¿está el código que esta operación necesita?
```

```python
def exigir_permiso(db, company_user_id: int, codigo: str) -> None:
    asignacion = (
        db.query(CompanyUserRoleModel)
        .filter(
            CompanyUserRoleModel.company_user_id == company_user_id,
            CompanyUserRoleModel.is_active.is_(True),
        )
        .first()
    )
    if asignacion is None:
        raise PermissionError("La membresía no tiene un rol activo")

    permitido = (
        db.query(RolePermissionModel)
        .join(PermissionModel)
        .filter(
            RolePermissionModel.role_id == asignacion.role_id,
            RolePermissionModel.is_active.is_(True),
            PermissionModel.code == codigo,
            PermissionModel.is_active.is_(True),
        )
        .first()
    )
    if permitido is None:
        raise PermissionError(f"El rol no tiene el permiso {codigo}")
```

`createTransaction` ya validaba monto, cuenta, concepto y empresa. Antes de hacer `commit`, llama `exigir_permiso(db, actor_id, "transactions.write")`. Si el rol es `CONSULTA`, el permiso no está, el resolver lanza el error y la fila no se guarda. GraphQL responde con `errors`. Si el rol es `CAPTURISTA`, el permiso sí está y el movimiento sigue el camino de siempre.

Las operaciones de esta unidad reciben `actorCompanyUserId`. Es la membresía con la que se está probando en GraphiQL. El servidor no cree el permiso que el cliente declare: busca el rol de esa membresía y lee la matriz. Pasar el id en el input alcanza para esta actividad. En la siguiente, el control de acceso tomará la identidad del token de `login` y el cliente ya no podrá elegir otra membresía.

`login` no cambia. Sigue devolviendo el token y los datos mínimos del usuario. La contraseña sigue guardada con hash. El tipo GraphQL `User` sigue sin exponer ese hash.

Actividad para entregar

Continúa el Sistema General de Ingresos de tu equipo. No abras un proyecto nuevo y no reescribas empresas, usuarios, cuentas ni movimientos. Agrega roles y permisos encima de lo que ya corre con Strawberry, MySQL y, si lo usan, Docker.

La entrega cubre dos temas de la unidad, en una sola carpeta:

1. Actividad #7. Diseño e implementación de roles y permisos en el sistema. 2. Configuración de permisos para diferentes roles del sistema.

```
ingresos/
  database.py
  models.py
  schema.py
```

Si el proyecto ya tiene esos archivos con otros nombres, se conservan. Lo que se entrega es el código que agrega las tablas, el esquema y las pruebas. El `docker-compose` anterior sigue levantando MySQL. No hace falta un contenedor nuevo.

Parte 1. Roles y permisos

Implementa `Role` y `Permission` como en las secciones 1 y 2.

- Las tablas son `roles` y `permissions`.
- Si están vacías al arrancar, inserta los cuatro roles y los diez permisos de este documento. Si ya tienen filas, no los dupliques.
- `roles` y `permissions` se consultan por GraphQL.
- `createRole` y `createPermission` insertan y hacen `commit`.
- `deactivateRole` no borra la fila.
- El código del rol y el código del permiso son únicos.

`isAdmin` y la regla de un solo administrador principal se quedan como están.

Parte 2. Permisos por rol y rol de la membresía

Implementa la matriz y la asignación.

- La tabla `role_permissions` une rol y permiso.
- Al arrancar, si un rol todavía no tiene permisos, carga la matriz de este documento.
- `rolePermissions(roleId:)` devuelve los permisos activos de ese rol.
- `assignPermissionToRole` rechaza una pareja duplicada.
- `removePermissionFromRole` desactiva la pareja.
- La tabla `company_user_roles` une la membresía con un rol.
- `assignRole` deja un solo rol activo por membresía.
- `createCompanyAdmin` deja `isAdmin` en true y además asigna el rol `ADMINISTRADOR`.

Parte 3. Una operación ya existente queda protegida

Elige las mutaciones que ya tienes y exige el permiso antes del `commit`:

Mutación	Permiso
`createCompanyUser` y `deactivateCompanyUser`	`company.users.write`
`createAccount`	`accounts.write`
`createConcept` y `assignConceptToAccount`	`concepts.write`
`createTransaction`	`transactions.write`
`assignRole` y `assignPermissionToRole`	`roles.write`

Las consultas de cuentas, conceptos y movimientos exigen el permiso `.read` correspondiente.

Cada una de esas operaciones recibe `actorCompanyUserId`. El permiso se decide con `exigir_permiso`, no con un `if isAdmin` nuevo.

Comprobación

Arranca el servidor como ya lo vienen haciendo (`strawberry` o FastAPI, según el proyecto) y abre `/graphql`. Guarda una captura por cada punto.

- `roles` muestra los cuatro códigos.
- `permissions` muestra los diez códigos.
- `rolePermissions` del capturista incluye `transactions.write` y no incluye `accounts.write`.

- `rolePermissions` de consulta no incluye ningún permiso que termine en `.write`.
- Con una membresía de capturista, `createTransaction` guarda el movimiento.
- Con una membresía de consulta, `createTransaction` responde `errors` y la tabla de movimientos no gana esa fila.
- Con una membresía de contador, `createAccount` funciona. Con el capturista, `createAccount` se rechaza.
- `assignRole` cambia a un usuario de capturista a contador. Después de eso, ese usuario sí puede crear una cuenta.
- Un segundo `createCompanyAdmin` de la misma empresa sigue fallando.

En MySQL, `SELECT` de `roles`, `permissions` y `role_permissions` debe coincidir con la matriz. Reinicia el servidor y confirma que los cuatro roles no se duplicaron.

Entrega

- El código del Sistema General de Ingresos con los modelos y el esquema nuevos.
- Captura de `roles` y de `permissions`.
- Captura de `rolePermissions` para capturista y para consulta.
- Captura de un movimiento aceptado y de un movimiento rechazado.
- Captura de `createAccount` aceptado para el contador y rechazado para el capturista.
- Captura, después de reiniciar, de que los roles no se duplicaron.
- Junto a cada captura, una frase: qué regla demuestra.

Criterios

- Los roles y los permisos son catálogos. No son columnas nuevas de `User`.
- La matriz vive en `role_permissions` y coincide con este documento.
- Cada membresía tiene un solo rol activo.
- El administrador principal conserva `isAdmin` y recibe el rol `ADMINISTRADOR`.
- Las mutaciones de la parte 3 consultan el permiso antes de escribir.
- Un rechazo no deja la fila a medias: no hay `commit` si el permiso falta.
- Los datos de una empresa no se mezclan con los de otra.
- Los nombres públicos de GraphQL coinciden con el esquema de este documento: `roles`, `permissions`, `rolePermissions`, `assignPermissionToRole`, `assignRole`.